Perceptron

1.What is a Perceptron?

A perceptron is the simplest fundamental unit of an artificial neural network — a single artificial neuron that takes in numeric input features, combines them via weighted summation, applies an activation function, and produces a binary classification output.

Running Example: To understand its mechanics intuitively, we use a running real-world example throughout these notes: predicting whether a student will Pass or Fail an exam based on hours of study (x1) and hours of practice (x2).

Mathematical Notation Reference

1. x1: Input 1 — hours of study
2. x2: Input 2 — hours of practice
3. w1, w2: Weights assigned to each input feature
4. b: Bias term (threshold adjustment)
5. z: Weighted sum ('net input') to the neuron (z = w1 x1 + w2 x2 + b)
6. f(z): Activation function (Step function)
7. y_hat: Predicted output (0 = Fail, 1 = Pass)
8. y: Actual ground truth target output
9. η: Learning rate (step size for parameter updates)

2. Architecture of a Perceptron

A perceptron network is structured into three distinct conceptual layers:

1. Input Layer: Receives raw numerical feature signals (x1, x2).

2. Hidden Layer: Contains the artificial neuron which computes the weighted sum and applies the activation function.

3. Output Layer: Produces the final binary decision prediction y_hat.

Figure 1.1: Perceptron Neural Network Architecture

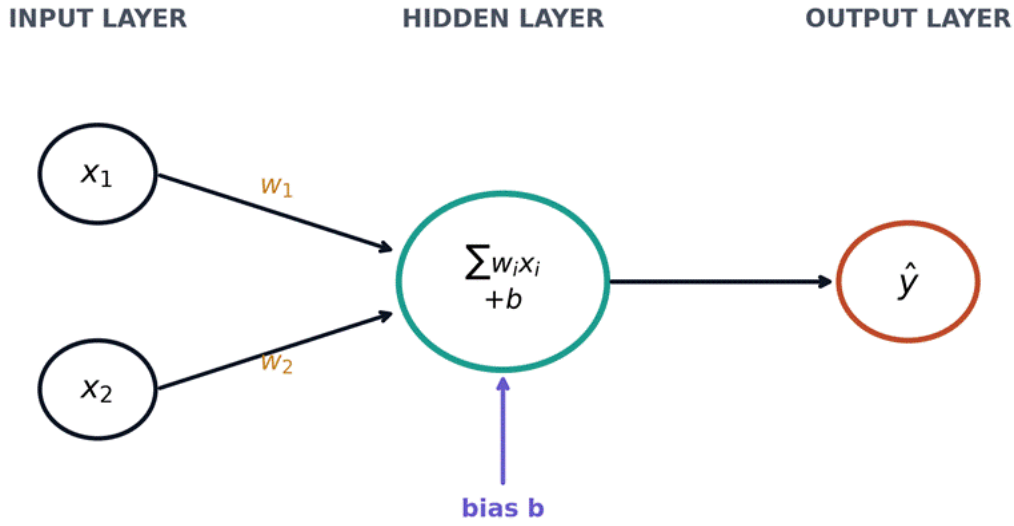


Figure 1.1: Input signals x1, x2 are scaled by weights w1, w2, summed with bias b inside the neuron, and passed through activation f(z) to produce y_hat.

Figure 1.2: Two-Stage Internal Processing Inside a Single Neuron

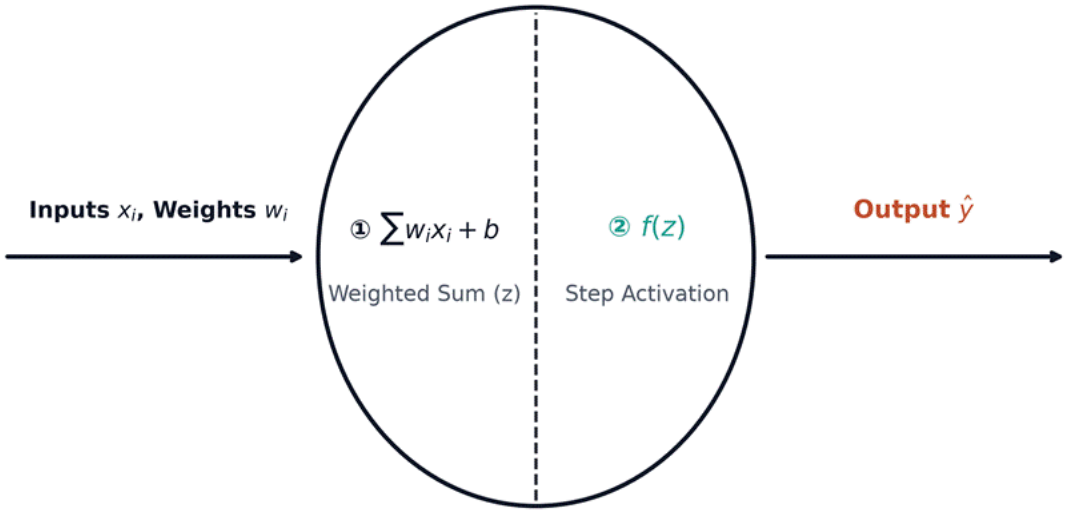


Figure 1.2: Two-stage internal neuron processing: ① compute weighted sum z, ② pass z through activation function f(z).

3. Forward Propagation Mechanics

The left-to-right flow of information from inputs to final output — weighted sum → activation → prediction — is called Forward Propagation.

3.1 Forward Propagation Mathematical Formulas

Forward Propagation Formulas

$$z = \sum_{i=1}^{n} w_i x_i + b, \quad \hat{y} = f(z)$$

3.2 Worked Solution: Forward Pass Computation

Step-by-step calculation evaluating a single neuron forward pass:

Worked Example: Perceptron Forward Pass Computation

Given Parameters:	$x_1 = 2, \quad x_2 = 3, \quad w_1 = 0.5, \quad w_2 = 0.8, \quad b = -2$
Compute Weighted Products:	$x_1 w_1 = 2(0.5) = 1.0, \quad x_2 w_2 = 3(0.8) = 2.4$
Compute Net Input z:	$z = x_1 w_1 + x_2 w_2 + b = 1.0 + 2.4 - 2.0 = 1.4$
Apply Step Activation f(z):	$f(1.4) = 1 \quad \text{since } 1.4 \geq 0$
Final Answer:	$\hat{y} = 1$ (Pass Decision)

Reading the Pipeline

Pipeline: x1, x2 -> weights (w1, w2) -> weighted sum z -> activation function f(z) -> output prediction y_hat.

This left-to-right processing pipeline is precisely what 'Forward Propagation' represents!

4. The Activation Function

The activation function determines whether the artificial neuron 'fires' (activates). The classic perceptron utilizes the Heaviside Step Function, converting the continuous weighted sum z into a crisp binary 0/1 decision.

4.1 Heaviside Step Function Formula

Perceptron Step Activation Function

$$f(z) = 1 \quad (\text{if } z \geq 0), \quad f(z) = 0 \quad (\text{if } z < 0)$$

Figure 3.1: Perceptron Heaviside Step Activation Function

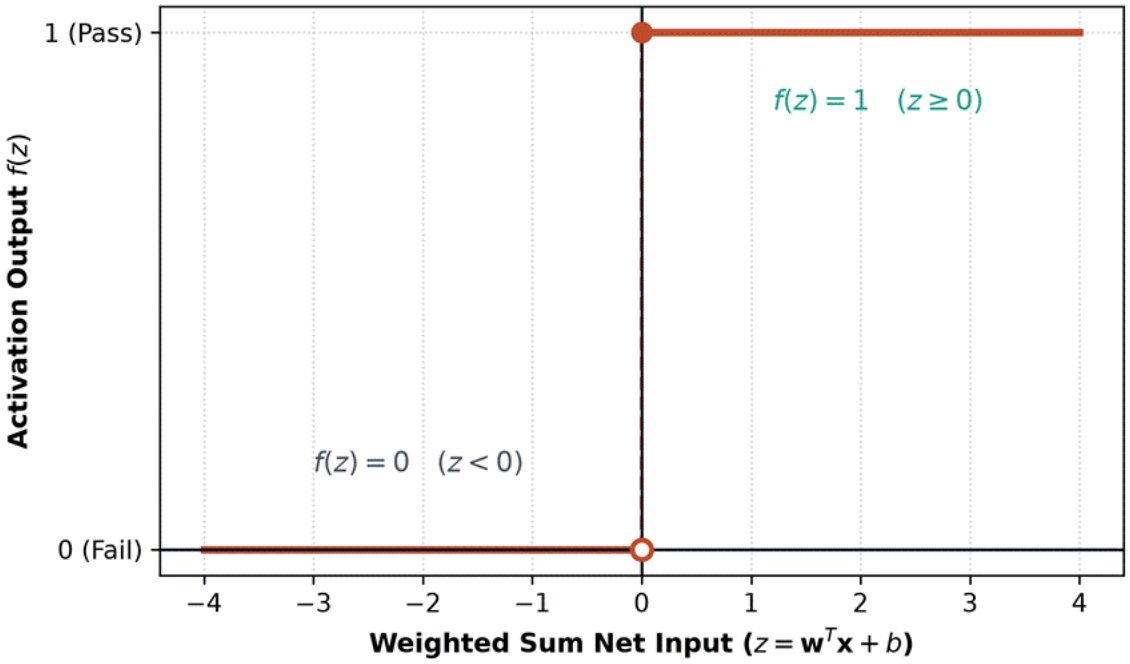


Figure 4.1: Step activation function plot jumping from 0 to 1 at threshold z = 0.

In our worked forward pass example: z = 1.4. Because 1.4 >= 0, f(1.4) = 1, yielding prediction y_hat = 1 (Pass).

5. Learning: The Perceptron Weight Update Rule

Forward propagation alone only computes a prediction — it does NOT learn. Learning takes place by comparing prediction y_hat with actual ground truth y, calculating prediction loss (y - y_hat), and adjusting weights and bias to eliminate error.

Learning Terminology

- y : Actual ground truth target
- y_hat : Predicted output
- Loss Error : (y - y_hat)
- eta : Learning rate parameter controlling correction step size
- xi : Input feature value

5.1 Weight and Bias Update Formulas

Perceptron Learning Weight & Bias Update Rule

$$w_i^{\text{new}} = w_i^{\text{old}} + \eta\,(y - \hat{y})\,x_i, \quad b^{\text{new}} = b^{\text{old}} + \eta\,(y - \hat{y})$$

5.2 Worked Solution: Weight Update Calculation

Step-by-step vertical calculation updating weights and bias when model under-predicts:

Worked Example: Perceptron Weight & Bias Update

Given Parameters:	$x_1 = 2, x_2 = 3, w_1 = 0.2, w_2 = -0.4, b = -1.0, y = 1, \hat{y} = 0, \eta = 0.1$
Compute Error (Loss):	$\text{Loss} = y - \hat{y} = 1 - 0 = +1$ (Under-prediction)
Update Weight w₁:	$w_1^{new} = 0.2 + 0.1(1)(2) = 0.2 + 0.2 = 0.4$
Update Weight w₂:	$w_2^{new} = -0.4 + 0.1(1)(3) = -0.4 + 0.3 = -0.1$
Update Bias b:	$b^{new} = -1.0 + 0.1(1) = -0.9$
Final Answer:	$w_1 = 0.4, \quad w_2 = -0.1, \quad b = -0.9$ (Weights Nudged Positive!)

Notice the correction direction: because the model under-predicted (y_hat = 0 when true y = 1), each weight is nudged positively in proportion to its corresponding input feature value.

6. Gradient Descent: Finding Optimal Weights

Iteratively repeating weight updates across training examples is analogous to walking downhill on a loss landscape: each update step moves parameter weights in the direction of steepest loss reduction until reaching minimum error.

Figure 4.1: Gradient Descent Optimization on Loss Landscape

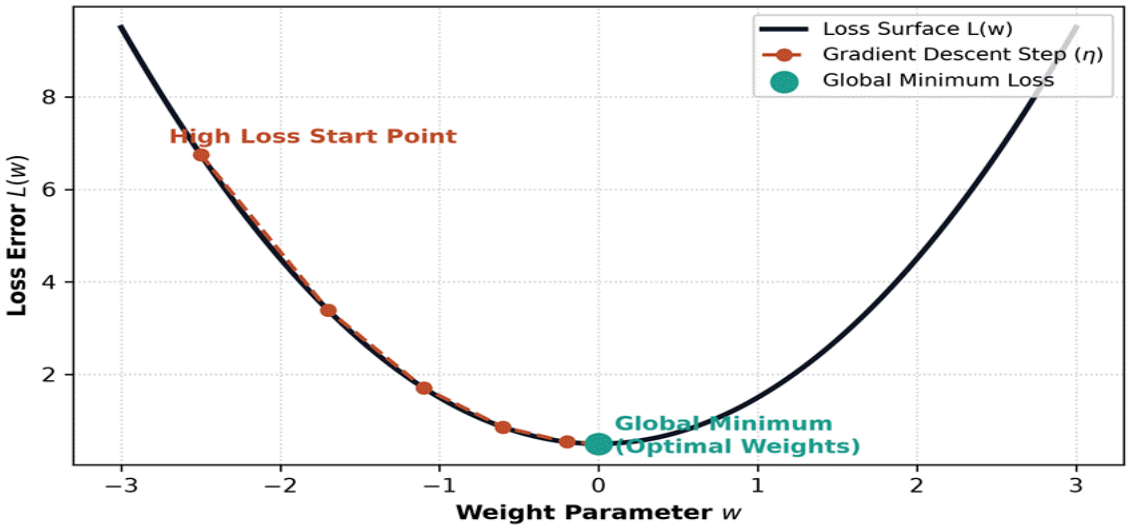


Figure 6.1: Gradient descent optimization stepping downhill on loss curve toward global minimum.

6.1 Impact of Learning Rate (eta)

Learning Rate Trade-offs

1. Learning Rate Too Small (eta -> 0): Convergence is extremely slow — tiny steps take thousands of iterations to reach minimum loss.

2. Learning Rate Too Large (eta >> 0): Steps overshoot the minimum, causing training loss to bounce erratically or diverge.

3. Local vs Global Minimum: A properly tuned learning rate ensures parameter weights converge toward the true global minimum rather than becoming trapped in a shallow local minimum.

7. Linear Separability: What a Single Perceptron Can (and Can't) Learn

A single-layer perceptron with a step activation draws exactly ONE straight decision boundary line (w1 x1 + w2 x2 + b = 0) in feature space. Consequently, it can only learn functions that are Linearly Separable.

7.1 AND Gate Truth Table (Linearly Separable)

x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

7.2 OR Gate Truth Table (Linearly Separable)

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1

7.3 XOR Gate Truth Table (NOT Linearly Separable)

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

Figure 5.1: Linear Separability Limitations of a Single Perceptron

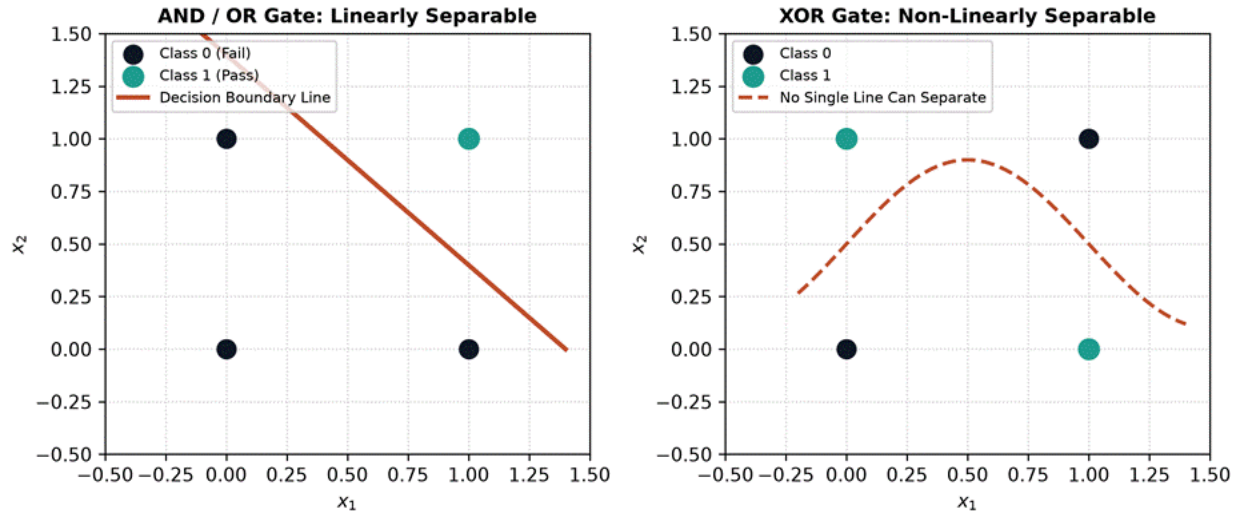


Figure 7.1: AND/OR inputs fall on opposite sides of a single straight line. XOR positive and negative targets are interleaved diagonally, making separation by a single straight line mathematically impossible.

Why Linear Separability Matters in Deep Learning

This fundamental limitation (proved by Minsky & Papert in 1969) is precisely why Multi-Layer Perceptrons (MLPs) with hidden layers and non-linear activation functions are required!

Stacking perceptrons in deep architectures allows the network to bend decision boundaries into complex curves, rendering non-linear problems like XOR completely solvable.

8. Summary & Revision

Key Takeaways

1. Forward Propagation: z = sum(wi xi) + b, then prediction y_hat = f(z).

2. Step Activation: Outputs 1 if z >= 0, else 0.

3. Prediction Loss Error: Calculated as (y - y_hat) in {-1, 0, +1}.

4. Weight Update Rule: wi_new = wi_old + eta(y - y_hat)xi, and b_new = b_old + eta(y - y_hat).

5. Gradient Descent: Iterative updates steer weights downhill toward global minimum loss.

6. Core Limitation: A single perceptron only solves linearly separable problems (AND, OR) — NOT non-linear problems (XOR).